kafka-goose-game

Implementation Documentation

July 2026

Overview

This is the full, top-down description of how the project is built and *why* it is built that way. It describes the system layer by layer. For each layer it gives the technical choices and the reason behind them, the design patterns used and the ones deliberately avoided, and the problems met along the way.

Terms that are standard in Kafka, Java or software design but not obvious on first reading are collected in chapter 11, each with a short explanation and a link to a source.

What this project is

A multiplayer **Game of the Goose** (Gioco dell'Oca) implemented in **plain Java 21** with the raw `kafka-clients` library — no Spring, no Kafka Streams, no framework of any kind. It is a test bed for two current stacks, put under load together:

1. **Kafka 4.x through the low-level client APIs**: KRaft with no ZooKeeper, keying and partition ordering, consumer groups against manual assignment, offset control, replay, delivery guarantees, failure modes. Driving the clients directly is what makes those choices visible, instead of settled by a framework's defaults.
2. **Java 21 as the design language**: records, sealed interfaces, exhaustive pattern matching, virtual threads, text blocks, immutable collections — used as the *primary* structure of the code, not as extras added at the end.

The question the project answers is how those features hold up when they carry a whole system: a three-broker cluster, an authoritative server, several clients, and a failure injected on purpose.

The result is a playable game: a 3-broker Kafka cluster in Docker, one authoritative server process, and any number of terminal clients that join, roll dice, and watch an ANSI board update in real time.

The one idea everything follows from

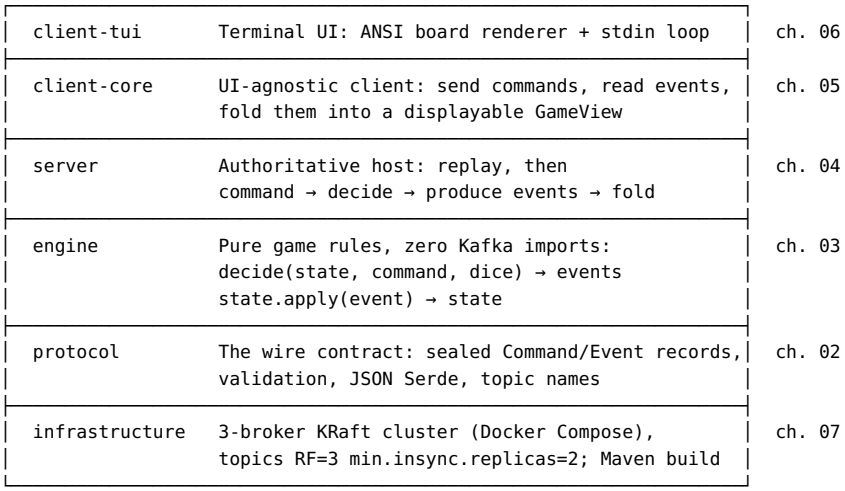
> ***The Kafka log is the only state. Everything else is a cache.***

The system is **event-sourced** and **server-authoritative**:

- Clients never change state. They publish *intents* (`Command`s) to the `game.commands` topic.
- One server is the single authority. It turns commands into *facts* (`Event`s) via a pure rules engine and appends them to `game.events`.
- Every piece of state held in memory is a **fold** over the event log: the events are applied one by one, in order, and the result is the current state. This is true of the server's game map and of every client's view of the board. Kill any process and restart it: it rebuilds itself by reading the topic again from the beginning.

Every design decision in the following chapters is a consequence of taking this idea seriously.

The layers, top-down



Dependencies point strictly downward, and they deliberately *skip* layers where that keeps the coupling low. `client-core` depends on `protocol` only, and **not** on `engine`, so a client never has the game rules available to it. Chapter 5 explains why.

Reading guide

Chapter	Contents
01 — Architecture	Event sourcing over Kafka: topics, keys, ordering, delivery guarantees, who is allowed to decide what, and the patterns that follow
02 — Protocol	The message types, why they are checked at the boundary only, the hand-written JSON Serde, and how it is protected against bad input
03 — Engine	The board rules, the split between deciding and applying, why goose chains always end, and the change to the deadlock rule
04 — Server	Replay at startup, the command loop, one thread that owns the state, at-least-once delivery, unreadable messages, clean shutdown
05 — client-core	The client library: the event loop on a virtual thread, replay on start, and why the fold is written twice on purpose
06 — client-tui	The terminal UI: rendering as a pure function, the snaking board layout, and input and output kept in one class
07 — Infrastructure & build	The KRaft cluster, the topic settings, and the Maven multi-module build
08 — Testing	What is tested at each layer, the repeatable end-to-end test, and what running the real system found that unit tests missed
09 — Patterns & anti-patterns	The full list in one place: every pattern used and every anti-pattern avoided, with pointers to where
10 — Implementation plan	The 8-step build order the project was actually executed in, one self-contained step per session
11 — Glossary	The Kafka, design, Java and testing terms used in these chapters, each explained in a few lines with a link to a source

Each chapter closes with a **Decisions** and an **Issues** section, the in-context version of the two project logs:

- `DECISIONS.md` — every non-obvious choice, with reasoning
- `ISSUES.md` — every problem hit, what was tried, what fixed it

To run the system (quickstart, TUI commands, the Kafka experiments to try against a live cluster), see the top-level README. The step-by-step build order the project followed is in chapter 10.

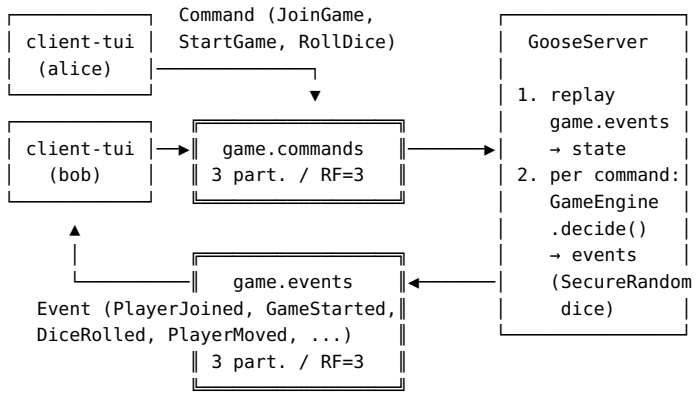
Fixed constraints the project was built under

These were decided up front (see chapter 10) and never revisited:

Constraint	Value	Motivation
Language / stack	Plain Java 21 + `kafka-clients` 4.3.0	Every Kafka decision stays explicit instead of being made by a framework
Serialization	JSON via Jackson, hand-written `Serde`	A log that can be read straight off the topic; no Schema Registry to operate
Cluster	3 KRaft brokers, topics RF=3, `min.insync.replicas=2`	Enough replication to *demonstrate* broker-loss survival, small enough for a laptop
Architecture	Event-sourced, server-authoritative, topics keyed by `gameId`	The design under test: everything else follows from it
UI	Terminal first, `client-core` kept UI-agnostic	A future web/desktop UI must be able to reuse the client layer unchanged
Tests	JUnit 5 everywhere; Testcontainers for the E2E	`mvn test` must never require Docker; `mvn verify` proves the real stack

01 — Architecture: Event Sourcing over Kafka

The shape of the system



Two topics, one direction each:

Topic	Written by	Read by	Meaning
`game.commands`	clients	the server	**Intents** — requests that may be rejected
`game.events`	the server only	the server (replay) and every client	**Facts** — things that irrevocably happened

This split between commands and events *is* the architecture. The rest of this chapter is about keeping each half strict: commands that can be refused, events that can be trusted.

Why event sourcing, and why here

Event sourcing is more than most projects need. It was chosen here because no other architecture puts so many Kafka mechanics under load at once, on a domain small enough that none of them are hidden by the business rules:

- **The log as source of truth** — Kafka's actual data model, not a queue metaphor. State is derived, disposable, and rebuildable (`replay`).
- **Ordering** — a game is only correct if its events are handled in order, and Kafka orders messages inside one partition only. That forces you to understand keys.

- **Delivery guarantees** — “was my event written exactly once?” stops being a question in a manual and becomes a bug you can cause yourself.
- **Several independent readers** — the server and any number of clients read the same `game.events`, in different consumer groups and for different reasons.

A board game is an ideal domain for it: turns are inherently sequential events, state is small, and rules are pure logic — so the *infrastructure* concepts stay in focus.

Server-authoritative: the trust model

Only the server writes `game.events`, and only the server rolls dice (`SecureRandom`, chapter 4). The design assumes clients cannot be trusted, and does not rely on them behaving well:

- A client cannot move a token — there is no command for it. It can only *ask* to join, start, or roll.
- An out-of-turn or otherwise invalid command produces **no events**; the server logs the rejection and moves on (chapter 3).
- A malicious or buggy client publishing garbage to `game.commands` is contained at two boundaries: the deserializer (payload cap, closed type set, field validation — chapter 2) and the engine (phase/turn checks).

There is one thing the demo cluster does *not* check: the brokers accept a write from anyone, so any process could write to `game.events` directly. This is a deliberate limit on the scope of the project. The cluster stays on PLAINTEXT so that every command-line experiment works without setting up credentials first. The README describes the missing piece — SASL/SCRAM with access rules, letting clients only write commands and only read events — as the next step rather than a missing one.

Keying, partitions, and ordering

Both topics have **3 partitions** and every record is **keyed by `gameId`**. Consequences, in order of importance:

1. **Per-game total order.** All events of one game land in one partition, so every consumer sees that game’s history in exactly the

order the server decided it. Cross-game order is not guaranteed — and doesn't matter, because games are independent.

2. **Per-game single writer at scale.** Commands for one game also hash to one partition, so if the server were ever scaled to multiple instances in one consumer group, each game would still be processed by exactly one instance — no distributed locking needed. (The current server is deliberately single-instance; the caveat is recorded in chapter 4.)
3. **Room to grow.** Three partitions are enough to show points 1 and 2, and they match the three brokers. Nothing more was needed.

The fold (`GameState.apply`, `GameView.apply`) filters or groups by `gameId`, so consumers reading the whole topic stay correct even though partitions interleave different games.

State is a fold — everywhere, identically

There are three folds in the system, and the design depends on all three producing the same result:

Fold	Where	Purpose
`GameState.apply(Event)`	engine, used by server	Authoritative state for rule decisions
`GameState.apply(Event)`	inside `GameEngine.decide`	The engine folds its *own* freshly-decided events before computing the next turn — decision logic and state logic share one source of truth
`GameView.apply(Event)`	client-core	Displayable state for UIs (adds a recent-event log)

`GameView` writes the fold again instead of reusing `GameState`, on purpose. The reasons — keeping dependencies clean rather than avoiding repetition, and why the message format is the real contract — are set out in chapter 5.

The fold **trusts the log**. `apply` does not check rules that span several events — for example, that a `TurnStarted` names a player who actually joined. The engine already guaranteed that when it decided the event, and only the server ever writes events. Checking it again here would mean keeping the same rules in two places, which is exactly the kind of repeated business logic the project's code style rules forbid. What `apply` *does* check is each event on its own: every

record validates its own fields in its compact constructor, and `apply` refuses events that belong to a different `gameId`.

Delivery guarantees: at-least-once, with the limits stated

The server processes commands **at-least-once**. The weakness that comes with that choice is written down rather than hidden:

- Producer: `acks=all` plus `enable.idempotence=true`. Once the broker confirms an event, that event is on at least 2 of the 3 copies, and a network retry cannot have written it twice.
- The command consumer's offsets are committed **only after** every produced event is confirmed (`flush()` then `Future.get()` before `commitSync()`).

So if the server crashes after writing events but before committing offsets, it reads those commands again on restart. For most commands this changes nothing: the engine simply rejects them a second time, and a repeated `JoinGame` for a player who already joined produces no events. A repeated `RollDice` is different — it **rolls again**, because the dice are random and the same state can give a different result.

Real exactly-once processing would need Kafka transactions, so that producing events and committing offsets happen as one atomic step. That was left out on purpose: transactions are the wrong complexity for this system, and the trade-off is recorded in `DECISIONS.md`. What matters is that the gap is *known and accepted*, and documented where it is created, rather than discovered later by someone reading a stack trace.

Clients need no delivery guarantees at all: they never commit offsets, they replay from the beginning on every start, and folding is idempotent from a fixed starting state.

Replay: three consumers, three offset strategies

The same topic is read three different ways, which together cover most of Kafka's consumer-offset design space:

Consumer	Group	Offsets	Why
Server replay (startup)	none — manual `assign` + `seekToBeginning`	never committed	Replay must *always* read everything, and a consumer group works against that: it would remember a position and reassign partitions
Server command loop	durable group `goose-server`	committed manually after produce-confirm	The one consumer whose position *is* meaningful state
Every client	a new group per run, `goose-client-<uuid>`, with `auto.offset.reset=earliest`	never committed	Every client start reads the whole history; the group exists only because `subscribe()` requires one

Failure containment

- **Poison pills** — records that cannot be deserialized — are logged and skipped, by moving the consumer past that offset. Both the server and the clients do this. A consumer loop must never die because of one bad record, because on restart it would read the same record and die again.
- **Broker loss** — with three copies of each partition and `min.insync.replicas=2`, any one broker can stop without losing data and without stopping the game. This was checked by playing a full game with one broker down (chapter 8). If a second broker is lost, `acks=all` writes start failing, and they fail visibly: that is the durability promise doing its job, not a defect.
- **Listener/UI failures** on the client are caught and logged so a rendering bug cannot stop the event stream (chapter 5).

Patterns applied at this level

- **Event Sourcing** — state as a fold over an append-only log.
- **CQRS on a small scale** — commands (requests to change something) and events (records of what happened) travel on separate topics with separate types. There is no shared “message” parent class.
- **Single Writer** — one authority per game’s history (the server; per partition via keying).

- **Functional core, imperative shell** — `decide` and `apply` in the engine are pure functions; all input and output happens in the server and the clients (chapters 3–5).

Anti-patterns avoided at this level

- **Dual writes** — writing the same change to two places, which can then disagree. The server never stores state *and* events as two separate operations: its local map is always built from exactly the events it produced.
- **Database-as-message-bus / shared mutable state** — processes communicate only through the two topics; no process reads another's memory.
- **Trusting the client** — no client-computed moves, no client-side dice.
- **Hidden delivery guarantees** — the at-least-once gap is documented in the code that creates it, instead of claiming exactly-once.

Decisions in this chapter

From `DECISIONS.md`:

- A rejected command produces no events at all.
- At-least-once was chosen over exactly-once.
- Replay assigns partitions manually instead of joining a consumer group.
- In-memory state can always be thrown away and rebuilt.
- The server assumes it is the only instance running.
- Topic names live in `protocol` as shared constants.

Issues hit at this level

From `ISSUES.md`, issue #7: two players could trap each other in the well and the prison, and the game could never continue. It looked like a bug in the engine, but the real lesson was about architecture. With a log you can only add to, you cannot go back and "correct the data"; you can only change the rules from now on, and the events already written must still fold correctly under the new rules (see chapter 3).

02 — Protocol: the Wire Contract

The `protocol` module is the shared language of the system, and the wire contract between the two sides: it is the only code that both the server side (`engine`, `server`) and the client side (`client-core`, `client-tui`) depend on. It contains the message types, their validation, the JSON Serde and the topic names, and nothing else. It holds no game rules, and does no input or output beyond turning messages into bytes and back.

The message hierarchies

Two sealed interfaces, one per topic:

`Command` — a player's intent (`Command.java`):

Record	Fields	Meaning
`JoinGame`	gameId, player	Ask to enter the lobby
`StartGame`	gameId, player	Ask to close the lobby and begin
`RollDice`	gameId, player	Ask to roll on one's own turn

`Event` — a fact decided by the server (`Event.java`):

Record	Key fields	Meaning
`PlayerJoined`	player	Lobby entry accepted
`GameStarted`	players (turn order), firstPlayer	Lobby closed, play began
`TurnStarted`	player	Whose turn it now is
`DiceRolled`	player, die1, die2	The server's roll (informational — the moves carry the state change)
`PlayerMoved`	player, from, to, `MoveReason`	One movement segment; a single roll can emit several (goose chains, bounce)
`PlayerStuck`	player, square	Trapped on inn 19 / well 31 / prison 52
`PlayerFreed`	player	No longer trapped
`GameWon`	player	Landed exactly on 63; the game ends here

Every event also carries `gameId` and an `Instant timestamp`. `MoveReason` (`NORMAL`, `GOOSE`, `BRIDGE`, `BOUNCE`, `MAZE`, `DEATH`) makes each movement

segment self-describing, so clients can render *why* a token moved without knowing any board rules.

Why records and sealed interfaces

- **Records** compare by content, not by identity, and cannot be changed after they are built. That is exactly what a message on the network is. The end-to-end test compares whole events with `assertEquals`, which only works because two records with the same values are equal.
- **Sealed** interfaces fix the list of message types, and the compiler knows that list. Every `switch` over `Command` or `Event` in this codebase covers all cases and has **no `default` branch** (see exhaustive pattern matching). Adding a ninth event type breaks the build until the engine fold, the client fold and the terminal renderer all handle it. The compiler, not a checklist, tells you what still has to change.

How much each event should say: many small facts, not one big one

One roll could have produced a single large `TurnResolved` event, carrying the dice, all the movement, any traps and the next player. Instead it produces a *sequence* of small events: `DiceRolled`, one `PlayerMoved` per movement step, then `PlayerStuck` or `PlayerFreed` if they apply, then `TurnStarted` or `GameWon`. The reasons:

- each event means one thing and needs one case in the fold, so the folds stay simple;
- the log can be read move by move in a console consumer, which makes the system's behaviour observable without any tooling;
- a future UI can animate each step, because each step is already a separate event.

The price is that a turn is not written as one indivisible unit. That turns out not to matter: an incomplete sequence can only happen if the server dies while writing it, and when the command is delivered again the rest of the turn is produced (see chapter 4).

Validation at the boundary — and only there

Every field is checked in the record's **compact constructor**, with the checks that repeat collected in the package-private `Validation` class:

- `gameId`: 1–64 chars of `[A-Za-z0-9_-]`
- `player`: 1–20 chars of `[A-Za-z0-9_-]`
- `players` list: non-empty, each name valid, **no duplicates**, defensively copied with `List.copyOf`
- dice 1–6, squares 0–63 (0 = off-board start), timestamps non-null
- `GameStarted` also checks that `firstPlayer` is one of `players`. A rule that links two fields belongs in the record that holds them both

This gives one strong guarantee for the whole system: **an invalid message cannot exist as a Java object**. There is no way to build one — not in normal code, not when reading from Kafka, not in a test — because Jackson also goes through each record's main constructor. A `Command` or `Event` with a null field, a name that is too long, or a die showing 7, simply cannot be created. Everything further along — engine, server, clients — can therefore use messages without a single defensive null check, and does.

Allowing only the characters `[A-Za-z0-9_-]` in names is a security decision, not a matter of taste. Names appear in log files, in terminal output, and in the output of command-line tools. Limiting the characters at the boundary removes a whole family of injection attacks at once: fake log lines, hidden ANSI escape sequences that rewrite what the terminal shows, and characters with a special meaning to a shell or a file path. The alternative is escaping the same values correctly in every place they are printed.

The Serde: one class, three interfaces

`JsonSerde<T>` implements Kafka's `Serializer<T>`, `Deserializer<T>` **and** `Serde<T>` in one class that holds no state (`serializer()` and `deserializer()` both return `this`). A single instance therefore works with ordinary producers and consumers today, and would work with Kafka Streams later without changes. One instance is built per

message family: `new JsonSerde<>(Event.class)` or `new JsonSerde<>(Command.class)`.

Design points, each with its motivation:

A closed set of types — the heart of the security design

The `"type"` field in the JSON comes from `@JsonTypeInfo(use = NAME)` together with an explicit `@JsonSubTypes` list on each sealed interface. Jackson's **default typing is never switched on**. The difference matters: with default typing, an incoming message can name *any* class available to the program, which is the well-known Jackson attack that ends in remote code execution. With an explicit list, a message can name exactly these 11 record types and nothing else. The list also matches the sealed interface: the compiler enforces it inside Java, the annotation enforces it on the network.

Payload cap before parsing

`MAX_PAYLOAD_BYTES = 10 KiB`, checked on the raw bytes *before* Jackson sees them. Real messages are a few hundred bytes, so anything larger is either broken or hostile. Refusing it early puts a fixed limit on how much memory an attacker can make the server use for parsing.

Unknown fields are tolerated — explicitly

`FAIL_ON_UNKNOWN_PROPERTIES = false`, with a comment saying why and a test that fails if anyone changes it. This came out of a review (ISSUES.md #5): Jackson's default of `true` was in force *by accident*, chosen by nobody and covered by no test. In an event-sourced system the log is kept forever, so a newer producer has to be able to add a field without breaking every consumer that has not been updated yet. Accepting unknown fields is safe here only because of the checks in the compact constructors: an extra field that nobody knows about is ignored, but a required field that is missing still fails. The accepted downside is that a misspelled field name is ignored in silence.

Tombstones: null in, null out

Both `serialize(null)` and `deserialize(null)` return `null`. This breaks the project's own "never return null" rule on purpose, and the code says so in a comment: a Kafka tombstone *is* a message with a null value, and a Serde has to let it through unchanged. The code that receives it handles the null openly — the server treats a null command or event as "nothing to do".

Inline `Instant` handling

Timestamps are written as ISO-8601 strings by a very small `SimpleModule` defined inline (two anonymous classes), instead of adding the `jackson-datatype-jsr310` dependency: two short classes against a whole library for one type. The reader also handles the case where the value is not a string at all. For input such as `"timestamp": {}`, `p.getValueAsString()` returns null, so the code raises a proper `ctx.wrongTokenException(...)` rather than letting a `NullPointerException` escape — another finding from review.

What happens to a message that cannot be read

Every failure to deserialize — broken JSON, an unknown `"type"`, a check that fails inside a compact constructor, a payload that is too large — is wrapped in the protocol's own `DeserializationException`, keeping the original exception as the cause. The message text uses `e.toString()` and not `e.getMessage()`, because `getMessage()` can be null and because the class name of the exception is what identifies a poison pill in the logs (again, a review finding). Consumers catch Kafka's `RecordDeserializationException` and **move past** the bad record: log it, skip it, never stop (chapter 4).

Topic names live here too

`Topics.COMMANDS` and `Topics.EVENTS` are constants in the protocol module, because topic names are part of the wire contract: the server and the clients must agree on them just as they agree on field names. Before this, each side had its own copy of the string, which a review found. One known weak point is recorded in DECISIONS.md: the `init-topics` service in `docker-compose.yml` still

writes the names out in YAML, so renaming a topic means changing `Topics.java` *and* the compose file.

Patterns applied

- **Algebraic data types** — a sealed interface plus records as the message model: a closed list of shapes, each holding its own fields.
- **Parse, don't validate** — reading a message *is* validating it, so the rest of the program only ever sees values that are already known to be correct.
- **Fail immediately at the boundary**, with error messages that name the field at fault.
- **State the policy instead of inheriting a default** — the handling of unknown fields is set in code, explained in a comment and held in place by a test, so it is a visible choice rather than an accident.

Anti-patterns avoided

- **Jackson default typing** — the attack it enables is removed by design, not merely made harder.
- **Messages built out of plain strings** — no maps of strings, and no switching on a `"type"` field of raw JSON anywhere outside the Serde.
- **Checking the same values again further along** — validation happens exactly once, when the object is built.
- **Methods that return null** — the two tombstone nulls are the only exception, they are commented, and the Kafka contract requires them.
- **Losing the cause of an exception** — every wrapper passes the original exception on.

Decisions (from DECISIONS.md)

- Unknown fields in incoming JSON are ignored, on purpose and with a test.
- The Serde passes null values through, because Kafka tombstones need it.
- One class implements all three Kafka Serde interfaces.

- `Instant` support is written inline instead of adding a dependency.
- The set of message types on the wire is closed and listed explicitly.
- Payloads larger than 10 KiB are rejected before parsing.
- Deserialization failures get their own exception type.
- Names may contain only `[A-Za-z0-9_-]`.
- Topic names are constants in the `protocol` module.

Issues (from ISSUES.md)

#5 — Jackson's default handling of unknown fields was in force without anyone choosing it. It is now set explicitly, explained in a comment, and held in place by a test.

03 — Engine: the Pure Rules Core

The `engine` module is the functional core of the system: the complete rules of the Game of the Goose, with **no Kafka imports at all** and no input or output. It depends only on `protocol`. Everything in it is either a pure function or a value that cannot change. That is what makes the rest of the system easy to test, and what makes the end-to-end test give the same result every time.

Three types and one seam:

Type	Role
`Board`	Movement rules only: what one dice roll does to a token position
`GameState`	Immutable snapshot; `apply(Event)` folds one event into the next state
`GameEngine`	All the rules: `decide(state, command, dice)` returns the events a command causes
`DiceRoller`	An interface with one method — the seam where a test supplies a fixed sequence of rolls and the server supplies `SecureRandom`

The decide / apply split

The central idea of the engine is to keep the two halves of event sourcing apart:

```
decide :  GameState × Command × DiceRoller  →  List<Event>     (may reject: empty list)
apply  :  GameState × Event                 →  GameState        (total: never rejects)
```

- **`decide` is where rules live.** It checks phase, turn ownership, player-count limits, duplicate names; it consults the board; it is the only code that creates events. It is pure: timestamps come from an injected `Clock`, dice from the injected `DiceRoller`, so the same inputs always produce the same events.
- **`apply` is where each event gets its meaning.** It says what an event does to the state, and it *trusts the log*: it does not check rules that span several events, because events can only come from `decide`, through a topic only the server writes. Checking them again in the fold would mean keeping every rule in two places, and it would make replaying old but valid logs break

whenever a rule changed. The change to the deadlock rule, below, is exactly that situation.

One small detail carries a lot of weight: **`decide` folds its own output through `apply` before working out whose turn is next**. Once it has built the events for the roll, the moves and any trap, it runs `after = state; for (e : events) after = after.apply(e);` and only then calls `advanceTurn(after, …)`. So the turn logic always works on the state exactly as the log will record it. The rules and the fold cannot disagree, because the rules *use* the fold.

Rejections produce no events

An invalid command returns an **empty list**: a roll out of turn, a join after the game started, a name already taken, a start with only one player, a wrong `gameId`. There is no `CommandRejected` event and no exception. The reasons: the log stays a record of what actually *happened*, a rejection is not something that ever needs replaying, and the server only has to write a line in its own log. The cost is that a client gets no answer when its command is refused. For a terminal UI that redraws on every accepted event this is acceptable, and DECISIONS.md records it as the point to revisit if a future UI needs to show refusals.

This is also why receiving a command twice is usually harmless: applying a `JoinGame` or `StartGame` again, against the state it already produced, simply returns nothing.

Board: movement as data

`Board.resolve(from, roll)` returns the whole `List<Move>` that a roll causes. Each `Move(from, to, reason)` is one step, and they are in order, so the caller takes the final square from the last element. Movement is *described*, not carried out: the board changes nothing and produces no events, and `GameEngine` turns each step into one `PlayerMoved` event. This is why a single roll can produce several `PlayerMoved` events, and why the terminal UI can describe every hop separately.

The rules encoded: bridge 6→12, maze 42→39, death 58→1, geese {5,9,14,18,23,27,32,36,41,45,50,54,59} repeat the movement, overshooting 63 bounces back by the excess, landing exactly on 63

wins. The inn (19), well (31) and prison (52) deliberately do **not** appear in `resolve`: they don't move the token — trapping is *turn* logic and lives in `GameEngine`. Each layer owns one concern.

Why goose chains always end

The obvious rule — "landing on a goose moves you forward by the roll again" — never stops. From square 59 with a roll of 8 the piece goes past 63, bounces back to 59, which is a goose, hops *forward* 8 again, and repeats for ever (ISSUES.md #1, found while designing, before any code was written).

The rule that was implemented instead: a goose hop repeats the movement **in the direction the piece is currently going**, and a bounce off 63 *reverses* that direction. The `Cursor(pos, step)` record carries a step that can be negative, and the bounce sets it to `-|step|`. With that rule it can be proved that the chain always ends:

- while the piece moves forward, every hop increases its position, so the chain either leaves the goose squares or reaches the bounce — and the bounce can happen only once;
- after the bounce, every hop decreases the position, and a backward chain runs out of squares. On the standard board it stops by square 42, and an extra guard at 0 makes sure `resolve` can never return a square outside the board, even on a board laid out differently.

So every chain is finite: the position only ever moves one way, it changes direction at most once, and the board has a limited number of squares, so the chain must eventually land outside the goose squares.

Validation at the boundary, again

`resolve` throws `IllegalArgumentException` if `from` is outside 0–63 or `roll` is outside 1–12 (ISSUES.md #3, found in review). A roll of 0 on a goose square would repeat a hop of length zero for ever. Before the fix, no caller in `GameEngine` could actually reach that state — but only because the `DiceRolled` constructor happened to check the value first. A public method has to guard its own inputs: being safe only because of how today's callers happen to behave is not the same as being safe.

GameState: the immutable fold target

`GameState` is a record made of records: `gameId`, a `Phase` (`LOBBY`, `RUNNING` or `FINISHED`), `players` in the order they joined (which *is* the turn order), `positions`, `stuck` (which player is held on which square), and two `Optional` fields, `currentPlayer` and `winner`. The `Optional`s mean that "no current player" and "no winner yet" can be expressed without using null at all. The compact constructor copies every collection with `List.copyOf` and `Map.copyOf`, so a caller cannot change a state after handing it over. The record cannot be modified at any depth, and "changing" it means building the next one, through small private helper methods such as `withPosition` and `withStuck`.

`apply` is a `switch` over the sealed `Event` types that covers every case and has no `default`, so a new event type breaks the build here first. One event changes nothing on purpose: `DiceRolled` only reports the roll — the `PlayerMoved` events carry the actual change — so it returns the state unchanged.

GameEngine: turn flow and the traps

After a roll resolves, `decide` inspects the landing square:

1. **63** → `GameWon`; the game is over and there is no next turn.
2. **A trap** (inn, well or prison), *unless trapping would freeze the game* → `PlayerStuck`. If the trap is the well or the prison and someone is already held there, that player is released with `PlayerFreed`: these two traps **exchange prisoners**.
3. Otherwise the piece simply stays where it landed.

Then `advanceTurn` picks the next player, walking the rotation from the current one:

- players held in the **well or prison** are skipped; they wait until another player takes their place;
- a player at the **inn** who is met on the first pass is released with `PlayerFreed`, but is still skipped this once, so the inn costs exactly one turn. This is done with two passes: the first releases inn players, the second finds the first player who can actually take a turn. In a two-player game the opponent therefore rolls twice in a row, which is what the board game rule says.

`GameStarted` carries `firstPlayer`, and that already says whose turn it is, so no separate `TurnStarted` is written at the start of a game. Every fold treats `GameStarted` as both "the phase changed" and "the first turn begins".

The change to the deadlock rule

The traditional rules contain a real deadlock. With two players, if one is in the well and the other then lands in the prison, each is waiting for the other to free them, and neither ever can. During Step 4 this was written down as faithful to the original game, if harsh, and accepted as unlikely. It then **happened in the very first game played by hand** (ISSUES.md #7): alice fell into the well, bob hopped from 45 to 52 into the prison, and the game stopped.

The fix, decided by the user and recorded in DECISIONS.md: **the last free player is never trapped**. `freezesTheGame(state, lander, square)` cancels the trap when all three of these are true: (a) the square is one that holds a player until someone replaces them, (b) nobody is on it — if someone is, the two players exchange places and one of them ends up free, and (c) every *other* player is already held in the well or the prison. Players at the inn do not count, because the turn order frees them by itself. When the trap is cancelled, the player simply stays on the square, free.

Two details are worth noting as event-sourcing lessons:

- The new rule changes `decide` only. `apply` was not touched, so older logs — including the frozen game — still fold to exactly the same result. **You change the rules from now on; what is already in the log stays.**
- The branch in `advanceTurn` that handles "every player is trapped" by writing no turn at all can no longer be reached through `decide`. It is kept, with a comment, because logs written before the rule change still need it.

Patterns applied

- **Functional core** — the decision function is pure; everything that would make it unpredictable, time and randomness, is passed in as `Clock` and `DiceRoller`.

- **State machine** — a `Phase` field plus switches that cover every case; moves that are not allowed are refused, not turned into exceptions.
- **Command to event** — the writing half of CQRS, with the fold as the reading half.
- **Make invalid states impossible to express** — `Optional` instead of null, records that cannot be changed at any depth, sealed interfaces.
- **Strategy through a functional interface** — `DiceRoller` is the only test seam the project needs, and there is no mocking framework anywhere in it.

Anti-patterns avoided

- **Hidden calls to the outside world** — no `Instant.now()` and no `new Random()` inside the rules. The same inputs always give the same output because of how the code is built, not by luck.
- **The same business rule written twice** — the rules exist once, in `decide`, and the fold trusts them. The fold that *is* repeated, in client-core, is a different trade-off; see chapter 5.
- **Loops that may never end** — the two possible infinite loops, the obvious goose rule and a step of zero, were removed: the first by proving the chain ends, the second by checking the arguments.
- **Using exceptions for normal control flow** — a refused command is an empty list, not a thrown exception. Exceptions are kept for programming mistakes, such as nulls and invalid arguments.

Decisions (from DECISIONS.md)

- A refused command produces no events at all.
- A goose hop keeps the current direction, which is what makes chains end.
- The inn costs exactly one turn.
- The well and the prison exchange prisoners; since 2026-07-03 the last free player is never trapped.
- `GameStarted` already says whose turn is first.
- `decide` folds its own events before deciding who plays next.

- `apply` trusts the log and checks nothing across events.
- `Board.resolve` validates its own arguments.
- `Clock` and `DiceRoller` are passed in, never created inside the rules.

Issues (from ISSUES.md)

#1 — the obvious goose rule loops for ever; found while designing. **#3** — `Board.resolve(x, 0)` could loop for ever; found in review. **#7** — the well and prison deadlock happened in the first game played by hand; the rule was changed and two tests now hold the new behaviour in place.

04 — Server: the Authoritative Host

`GooseServer` is the imperative shell around the engine, and the only process allowed to write to `game.events`. Its entire life is two phases inside a single `run()` call:

```
run():
  1. replayEvents()      — rebuild Map<gameId, GameState> from game.events
  2. processCommands()   — loop: poll game.commands
                                → engine.decide()
                                → produce events to game.events (keyed by gameId)
                                → fold events into local state
                                → commit offsets (only after produce confirmed)
```

It depends on `engine`, and through it on `protocol`, and is about 280 lines including configuration. Keeping the rules pure is what allows this outer layer to stay this small.

Startup replay: state is throwaway

On every start the server rebuilds *all* state by reading `game.events` from the beginning. There is no snapshot, no database, no local file. Consequences:

- a crash can never leave the stored state and the log disagreeing, because the log *is* the state;
- releasing a change to the rules is just a restart: the fold does not reinterpret anything, and what `apply` means stays the same — see the change to the deadlock rule in chapter 3;
- startup takes longer as the log grows. At the size of a board game that does not matter, and it is the standard trade-off event sourcing makes. Snapshots would be the next step, and are deliberately out of scope.

Replay uses a consumer with **no group**: it calls `assign()` on all partitions itself, then `seekToBeginning()`, with auto-commit off and positions never committed. A consumer group exists to *share progress* between readers, and replay needs the opposite: read everything, alone, every time. The server knows it has finished by reading `endOffsets()` first and polling until every partition's `position()` has reached it.

Before assigning anything, the server calls `partitionsFor(Topics.EVENTS)`. If the topic is missing it stops at once with a clear `IllegalStateException` asking whether the cluster is up and `init-topics` has run. Kafka is configured never to create topics automatically (chapter 7), so a missing topic means the environment is only half started. Whoever is running it should be told immediately, instead of watching a replay that silently finds nothing.

The command loop: at-least-once, in the right order

Per poll batch:

1. Every command goes through `handleCommand`, which fetches the game's state or creates it. It uses `computeIfAbsent`, which looks up and inserts in one step, instead of checking first and then inserting. Then it calls `engine.decide(state, command, dice)`.
2. Rejected commands (empty event list) are logged and dropped.
3. If the command is accepted, each event is sent with `producer.send(...)`, using `event.gameId()` as the key, and the returned futures are kept. The state is folded straight afterwards with `applyEvent`, so the local map is only ever built from exactly the events that were sent to the log.
4. When the batch is done: `producer.flush()`, then `Future.get()` on **every** pending send. Any failure to write then appears as an exception that stops the batch *before* step 5.
5. Only then `consumer.commitSync()`.

The order of steps 4 and 5 is the whole delivery story. Offsets are never committed for commands whose events might not have reached the log. A crash anywhere before step 5 means those commands are delivered again. The engine refuses most of the repeats, but a repeated `RollDice` rolls new dice: that is the known at-least-once gap (chapter 1).

Producer settings: `acks=all` and `enable.idempotence=true`. A confirmed write is on at least 2 copies, because `min.insync.replicas=2`, and Kafka's own retries cannot write it twice. Consumer settings: the lasting group `goose-server`, auto-commit **off**, and `auto.offset.reset=earliest` so that a brand-new group starts at the first command. Auto-commit is off because it commits on a timer,

which could commit *before* the events were written — which is precisely the bug the manual ordering above exists to prevent.

Threading: single-threaded by design

The thread that calls `run()` owns *everything*: both consumers, first for replay and then for commands, the producer, and the `Map<String, GameState>`. There is no lock anywhere in the file because nothing is shared. Keeping all the data inside one thread is the concurrency strategy here, chosen on purpose instead of locking.

The one concession to the outside world is shutdown, and it follows Kafka's own rules:

- `KafkaConsumer` is *not* thread-safe; its single thread-safe method is `wakeup()`.
- `close()` (callable from any thread — the JVM shutdown hook, the E2E test) therefore only **signals**: it sets `volatile boolean running = false`, calls `wakeup()` on whichever consumer is active (an `AtomicReference` updated as `run()` moves between phases), and awaits a `CountDownLatch` with a 10-second bound.
- The run thread notices (`WakeupException` from `poll`, or the flag), exits its loop, and **closes its own resources** on the way out — try-with-resources in the same thread that opened them. A `started` flag keeps `close()` from waiting on a server that never ran; the latch counts down in `run()`'s `finally`, so `close()` is bounded even if the loop died of an exception.

`volatile` is used only for what it is safe for: making one variable's new value visible to another thread. It is never used for read-then-write sequences, which it does not protect.

Dice: `SecureRandom`, server-side only

`main()` connects `DiceRoller` to a `SecureRandom`, behind the `RandomGenerator` interface. Clients cannot roll at all, and the server's rolls cannot be predicted from earlier ones, which is possible with a seeded `java.util.Random`. For a board game this is more care than strictly needed, but a secure default that costs one line is not worth arguing about, and the seam is the same one the tests use. The end-

to-end test supplies a fixed list of rolls through the same seam. `main` is the only place in the system where randomness exists at all.

Messages that cannot be read

Both consumers poll through a small wrapper. It catches Kafka's `RecordDeserializationException`, logs the partition, the offset and the cause, and then calls `seek(partition, offset + 1)` to step over exactly that one record. Without the `seek`, the next `poll()` returns the same record again and the loop turns into an endless crash-and-retry cycle caused by a single hostile message — a poison pill. A tombstone, meaning a null command or event, is handled just as openly as "nothing to do". The server has to survive anything a client can put on `game.commands`.

The single-instance assumption

Commands are keyed by `gameId`, so several servers in one consumer group would divide the games neatly between them: each game handled by exactly one server, with no coordination needed. The problem is elsewhere. After the startup replay, each server folds only the events of the partitions it was given, so its copy of the other servers' games would slowly fall out of date. That is harmless, because it never acts on those games, but it wastes memory and confuses anyone reading the state. Doing this properly would mean replaying only the assigned partitions and handling the moment when Kafka reassigns them. DECISIONS.md records this as a limit that was chosen and written down, rather than one nobody noticed: for now, run a single server.

Patterns applied

- **Imperative shell** around a pure core — all the Kafka code, all the threading and all the logging live here, and none of the game rules do.
- **All state inside one thread**, with a shutdown that only signals and then waits, through `wakeup()`, the one method that may be called from another thread.
- **Do not acknowledge the input until the output is safely stored** — waiting for the events to be confirmed before com-

mitting offsets is the same ordering as the transactional outbox pattern, done with plain Kafka features.
- **Stop immediately on a broken environment** (a missing topic) but **contain a bad message** (a poison pill). Opposite responses, each matched to the kind of failure.

Anti-patterns avoided

- **Auto-commit while doing real work** — the default `enable.auto.commit=true` quietly turns this workload into at-most-once. It is switched off and replaced with an explicit order of operations.
- **Sending and not checking** — every `send` is confirmed, so a failed write stops the batch instead of disappearing.
- **State shared between threads** — there is no state map behind a lock; the state stays in one thread instead.
- **Consumer loops that die on bad input** — handled by stepping past the bad record.
- **Ignoring `InterruptedException`** — the interrupt flag is set again and the exception is passed on, in both `awaitAll` and `close`.
- **Unclear ownership of resources** — whoever opened a resource closes it, and `close()` called from another thread never touches a Kafka client except through `wakeup()`.

Decisions (from DECISIONS.md)

- One thread owns all the state.
- At-least-once was chosen over exactly-once, with the reasoning written down.
- Replay assigns partitions by hand instead of joining a group.
- Local state can always be thrown away and rebuilt from the log.
- The server assumes it is the only instance running.
- The end-to-end test gives the same result every time, through the `DiceRoller` seam.

Issues (from ISSUES.md)

#4 — Testcontainers against Docker daemon 29. The docker-java client inside Testcontainers asked for API version 1.32, which the

daemon refused. Setting an environment variable did not help, and neither did overriding the dependency, because the client is *shaded*: it is repackaged inside Testcontainers under different class names, so a normal dependency override never reaches it. The fix was the system property `api.version=1.44`, set permanently in the failsafe configuration.

#6 — `kafka-clients` does not make slf4j available at compile time, so every module that logs declares `slf4j-simple` for itself.

05 — client-core: the UI-Agnostic Client Library

`client-core` is the layer that makes a new user interface cheap to write. It holds everything a client needs in order to take part in a game — sending commands, following events, folding them into state that can be displayed — with **no console input or output and no assumptions about the UI**. A web or desktop interface would depend on this module in exactly the same way the terminal UI does, by implementing one interface.

Three types:

Type	Role
`GameClient`	Connection: command producer + event-loop consumer on a virtual thread
`GameView`	Immutable displayable state: the client-side fold, plus a recent-events log
`GameListener`	The UI contract: `onEvent(Event)`, `onViewUpdated(GameView)`

Dependency: **`protocol` only.** Not `engine` — deliberately.

The deliberately duplicated fold

`GameView.apply(Event)` writes the event fold again instead of reusing the engine's `GameState`. This is the decision in the codebase that is easiest to argue with, so the reasoning is set out in full.

Why write it twice?

- The implementation plan fixes which module may depend on which: a client must have *no game rules* available to it. `GameState` lives in `engine`, and depending on `engine` just to reuse the fold would bring `Board` and `GameEngine` along with it. Every future UI would then be able to decide game rules, which is exactly what clients must never do.
- The two folds are there for different reasons and had already started to differ. `GameView` also keeps `recentEvents`, a short list for display, and will grow more conveniences meant for the screen, which have no place in the engine.

- **The message format, not a shared class, is the contract.** Server and client agree because they use the same sealed `Event` types, and both folds are switches that must cover every case. A new event type breaks *both* builds until both folds handle it, and the shared protocol tests check the rest.

What it costs: the same fold logic exists twice, about 100 lines, and if the two ever behaved differently, a client would draw a board the server does not agree with. The cost was accepted knowingly and recorded in DECISIONS.md. This is the judgement call about coincidental duplication: the two folds are *meant* to grow apart, and that is exactly when merging them into one shared piece of code is the wrong move.

`GameView` is built with the same discipline as `GameState`: a record, unable to be changed at any depth thanks to `List.copyOf` and `Map.copyOf` in the compact constructor, `Optional` instead of null, a switch that covers every case, and small private helpers to build the next value. `recentEvents` holds at most 10 entries (`RECENT_EVENTS_LIMIT`) and drops the oldest first. It is a list for the screen, not a history: the history is the topic.

Replay-on-start: the client keeps nothing

Every `GameClient` start creates a consumer with a **fresh group id** (`goose-client-<uuid>`), `auto.offset.reset=earliest`, auto-commit off, and offsets that are never committed. So a client started, or restarted, in the middle of a game rebuilds its whole view by reading `game.events` from the beginning. Killing a client and watching the board come back is not a feature added on top: it is simply what a read model does. The group id exists only because `subscribe()` demands one, and nothing is ever stored under it.

Events are filtered by `gameId` in the client, because all games share the topic, and tombstones are skipped.

Threading model

- **The event loop owns the consumer and the fold.** It runs on a virtual thread (`Thread.ofVirtual()`). The poll loop spends nearly all its time waiting for the network, which is exactly what virtual threads are for; giving each client a full operating-system

thread would waste one. The `view` field is `volatile` so that any thread can read the latest value through `view()`. That is safe because `GameView` cannot change: the only thing that has to work is making the new value visible.

- **Listener methods run on the event-loop thread, in log order.** The rule is written on `GameListener`: a UI that needs its own thread must move the work there itself. An exception thrown by a listener is caught, logged and passed over, because a drawing bug in a UI must not stop the flow of events (`notifyListener`).
- **Commands may be sent from any thread, and so may `close()`.** `KafkaProducer` is safe to use from several threads — the one Kafka client that is. Shutdown works like the server's: signal, do not touch. A `volatile running` flag, then `consumer.wakeup()` through an `AtomicReference`, then `eventLoop.join(CLOSE_TIMEOUT)` using Java 21's `join(Duration)`, with a warning in the log if the loop has not stopped after 10 seconds. The producer is closed by `close()` because that thread owns it: it was created in the constructor and only ever used through a thread-safe API.

The `connect()` static factory

`GameClient`'s constructor is private and does **not** start the event loop; it creates the virtual thread *unstarted*. The public entry is:

```
public static GameClient connect(String bootstrap, String gameId, GameListener listener) {
    var client = new GameClient(bootstrap, gameId, listener);
    client.eventLoop.start();
    return client;
}
```

The reason: starting a thread inside a constructor hands out `this` before the object is fully built. This is the well-known *this-escape* problem — the new thread can see final fields that are not set yet. The factory method starts the thread only after the constructor has finished. This came out of a code review and was fixed by changing the structure, not by silencing the warning.

Sending commands

`send()` adds a **callback** that logs any failure. The result of a send must never be thrown away in silence, because a client that does

not wait for an answer has nothing else to tell it that a command was lost — that log line is the only signal. (A review found that the original code dropped the result.) After each send the producer is flushed with `flush()`. At the speed a person plays, sending immediately is better than collecting messages into batches: a player's `roll` should be on its way before they lift their finger off the key.

Checking the input costs nothing extra: `client.join("bad name!")` throws `IllegalArgumentException` from the `Command.JoinGame` compact constructor *before* Kafka is involved at all. That is the protocol's parse, don't validate design working locally as well.

Events that cannot be read are skipped by stepping past them, exactly as in the server.

Patterns applied

- **Ports and adapters** — `GameListener` is the port, the terminal UI (or any future UI) is an adapter, and `client-core` knows about none of them.
- **Read model** — `GameView` turns the stream of events into a shape meant for display.
- **A static factory method instead of a public constructor**, to avoid the this-escape problem and to give the operation a name: `connect`.
- **One virtual thread per long-running I/O loop** — what Java 21 intends virtual threads for.
- **Publishing an unchangeable value** — a `volatile` reference to an immutable object, so readers get a consistent view without any locking.

Anti-patterns avoided

- **Letting `this` escape from a constructor**, by starting a thread before the object was finished — solved with the factory method.
- **Throwing away the result of an asynchronous call** — solved with the producer callback.
- **A UI exception stopping the flow of data** — the listener is isolated.

- **Keeping offsets on the client** — there is nothing to move and nothing to corrupt: a restart simply reads everything again.
- **Using one consumer from several threads** — it stays in one thread and is stopped with `wakeup()`, as in the server.

Decisions (from DECISIONS.md)

- `GameView` repeats the fold on purpose, for the reasons given above.
- Every client run uses a new group and reads the whole log from the start.
- Listener methods run on the event-loop thread, and this is documented.
- `connect()` is a factory method, so the constructor starts no thread.
- The producer is flushed after every command.
- `recentEvents` never holds more than ten events.

Issues (from ISSUES.md)

#6 — the first build failed. `kafka-clients` uses slf4j internally but does not make it available at compile time, so `slf4j-simple` is now declared explicitly, with its version managed by the parent POM.

06 — client-tui: the Terminal UI

`client-tui` is the smallest module, and it is the proof that the boundary drawn by `client-core` works: a complete user interface in two classes, depending on `client-core` and nothing else.

Class	Role
`BoardRenderer`	**Pure**: `GameView` → ANSI string. No I/O of any kind.
`Main`	All the I/O: args, stdin loop, printing, process lifecycle.

BoardRenderer: rendering as a pure function

`render(GameView)` returns the whole screen as a single string: the 63-square board, a status block with the players, their positions, whose turn it is, any traps and the winner, then the list of recent events and a legend. Keeping this function pure has one reason and one large benefit:

- **The reason:** rendering is arithmetic about positions, which is exactly the kind of code where off-by-one mistakes hide. That arithmetic is very easy to unit-test *as long as* no console is involved.
- **The benefit:** the tests remove the ANSI colour codes with a regular expression and then check plain text, such as `"55 "`, `"19I"`, `"12 ABCDEF"` and `"*** alice WINS! ***"`. Nine tests cover the snaking layout, the markers on special squares, where pieces are drawn, the status block, the event lines, and the handling of the ANSI codes themselves — without driving a terminal at all.

The snaking board

The traditional board winds back and forth. The grid is 7 rows by 9 columns, with square 1 at the bottom left. Counting rows from the bottom starting at zero, even rows run left to right and odd rows right to left, so square 63 ends up in the top-left area:

```
55  56  57  58X 59* 60  61  62  63
54* 53  52P 51  50* 49  48  47  46
37  38  39  40  41* 42< 43  44  45*
36* 35  34  33  32* 31W 30  29  28
19I 20  21  22  23* 24  25  26  27*
18* 17  16  15  14* 13  12  11  10
 1   2   3   4   5*  6>  7   8   9
```

Each cell is 8 columns wide: two for the number, one for the marker, then the players' initials and padding. The markers show what the square does — `*` goose, `>` bridge, `<` maze, `X` death, `I` inn, `W` well, `P` prison — and are coloured by kind: traps red, jumps cyan, geese green. Each player gets one ANSI colour, chosen by the order they joined, and their piece is the first letter of their name in bold capitals.

One edge case found in review is worth describing. Six players on the same square produce 9 visible characters inside a cell only 8 wide. The original padding calculation then called `" ".repeat(-1)`, which throws `IllegalArgumentException`: the renderer crashed on a game state that was perfectly *valid*. The fix is `Math.max(0, CELL_WIDTH - visible)`, with a comment explaining the choice — lose one column of alignment rather than throw — and a test that keeps it that way (`sixPlayersOnOneSquareStillRender`). Rendering code must produce a result for every state it is willing to accept.

Event narration

`eventLine(Event)` turns each event into one sentence a person can read, such as "alice rolled 3+4 = 7" or "bob moved 6 -> 12 (bridge)". It is another switch that must cover every case, so a new event type breaks this build as well. All the formatting fixes `Locale.ROOT`, both for the `%2d` cells and for the lower-cased `MoveReason`. The board output is read by the tests as text, and formatting that changes with the machine's language settings is a classic hidden bug — in Turkish, for example, the lower-case form of "I" is not "i", so a name or a keyword can quietly stop matching.

Main: where all the side effects live

`Main` owns everything impure, and nothing else:

- **Arguments** `[bootstrap [gameId [player]]]`, with the defaults `localhost:9092 game-1 <os-user>`. The operating-system user name is cleaned up to match the protocol's `[A-Za-z0-9_-]{1,20}`

rule: remove the characters that are not allowed, cut it to length, and fall back to `"player"` if nothing usable is left. The default has to be valid by construction, or the very first command the program suggests would throw.

- **The input loop** runs on the main thread and accepts `join [name]`, `start`, `roll`, `board`, `help` and `quit`. The reader is fixed to UTF-8. If an `IllegalArgumentException` arrives, because someone typed an invalid name, it prints `invalid: …` and carries on: a mistake by the user deserves a new prompt, not a stack trace.
- **What triggers a redraw**: the `GameListener` callback, running on the client's event-loop thread, clears the screen and prints it again on every update. Both threads write to `System.out`, so now and then the prompt appears in the middle of the board. That is the accepted price of a terminal UI built on plain standard output, and it is written down in the class javadoc instead of being "fixed" with a terminal library the project does not need.
- **Identity is an agreement, not a session**: `join bob` changes which player the following `start` and `roll` commands act as. It is kept in an `AtomicReference<String>` because the listener thread reads it while rendering. There is no authentication anywhere. That is a deliberate limit: proving who a player is has nothing to do with what this project teaches, and the README lists it as future work together with SASL.

Two environmental details with reasons:

- `org.slf4j.simpleLogger.defaultLogLevel=warn` is the first thing `main()` does. `kafka-clients` writes a great deal at INFO level, and those lines print straight over the board.
- ANSI escape characters appear in the source **only as `\u001B` unicode escapes**, never as raw ESC bytes. For a while the generated sources contained invisible 0x1B control characters. They survive copy and paste, they make diffs hard to read in ways that are easy to miss, and they are a problem for anyone maintaining the code. They were removed with `sed`, and the rule was written down in DECISIONS.md.

Rolling blindly: an unplanned benefit

Because the server refuses an out-of-turn `RollDice` with *no events and no error*, a client can be driven by the simplest possible script

— send `roll` once a second — and the game still plays correctly. This is what made the automated test games possible: two clients fed from a pipe, playing complete games through the real cluster (chapter 8). Nobody designed this; it followed from the decision that refused commands produce no events. It proved its worth almost immediately, by causing the well and prison deadlock in the first game played for real.

Patterns applied

- **Functional core, imperative shell, on a small scale** — the same split as engine and server, one layer higher: `BoardRenderer` is pure, `Main` is not.
- **Humble object** — `Main` is deliberately too thin to be worth testing, and all the logic that could actually be wrong sits in the renderer, which is easy to test.
- **Rendering that must cover every case** — the sealed interface forces the UI to keep pace with the protocol, and the compiler checks it.

Anti-patterns avoided

- **Logic in the input/output layer** — `Main` makes no decisions about the game or the layout.
- **Rendering code that fails on some valid inputs** — the negative-repeat crash was removed and a test keeps it away.
- **Output that changes with the machine's language settings** — every format call fixes `Locale.ROOT`.
- **Invisible control characters in source files** — the `\u001B` rule.
- **Reaching for a framework by reflex** — no curses or JLine dependency for a problem that plain standard output solves; the occasional interleaved prompt is documented instead.

Decisions (from DECISIONS.md)

- The renderer is a pure function.
- The board snakes, with the row directions described above.
- ANSI escapes are written only as `\u001B` in source.
- Kafka's log level is lowered before anything is printed.

- Rolling out of turn is safe, which makes scripted clients possible.
- `join` changes which player the next commands act as.

Issues (from ISSUES.md)

#7, the deadlock, was *found* through the test games played with this module and fixed in the engine. The crash on an overfull cell, and a test that claimed square 55 was a goose square when it is not, were both found during review and fixed before the commit.

07 — Infrastructure & Build

The Kafka cluster

`docker-compose.yml` in the root of the repository defines the whole running environment: three brokers plus one short-lived service that creates the topics. Each choice, with its reason:

KRaft, combined mode, three nodes

- **KRaft, so no ZooKeeper** — this is how Kafka 4.x manages itself, and ZooKeeper is on its way out. All three nodes run as **both controller and broker** (`KAFKA_PROCESS_ROLES: broker,controller`), and all three take part in the controller vote. That is fine for development and saves running a second group of three containers. A production cluster would keep the two roles apart, and seeing them combined here is what makes that difference visible.
- **Three brokers** is the smallest number that makes replication *interesting*. Three copies with `min.insync.replicas=2` keeps working when one broker is lost, and makes writes fail visibly when a second one goes. Both were shown on a running cluster (chapter 8).
- **No volumes, on purpose** — `docker compose down` leaves a completely fresh cluster. Being able to throw the state away is worth more here than keeping it: every experiment starts from the same known point, so a result means something.

Listeners: two networks, two addresses

Each broker offers two listeners for data, because a Kafka running in Docker is reached from two different networks under two different addresses:

- `PLAINTEXT://kafka-N:29092` — for traffic *inside* the compose network (inter-broker, the init job);
- `PLAINTEXT_HOST://localhost:9092/9094/9096` — advertised to *host* processes (the server and clients run on the host).

Getting `advertised.listeners` right is the most common problem people hit when running Kafka in Docker. The compose file lists both addresses in the comment at the top.

How the topics are created

- `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"`. The game topics are created explicitly by the short-lived `init-topics` service, with 3 partitions, 3 copies and `min.insync.replicas=2`. A misspelled topic name should be an error, not a topic quietly created with one copy and default settings.
- Kafka's own internal topics, such as `__consumer_offsets`, are also set to 3 copies and a minimum of 2 in sync. Losing a broker must not take out the place where *offsets* are stored either. Forgetting this is a common gap in development compose files.
- `init-topics` waits for all three brokers to report healthy, using a `kafka-broker-api-versions.sh` check. It then creates both topics with `--if-not-exists`, so starting the cluster again changes nothing, prints their description into the log, and exits with status 0. The ordering comes from Compose's `depends_on: condition: service_healthy`, with no sleep loops anywhere.

One known weak point (DECISIONS.md): the topic names appear both in `Topics.java`, where they are part of the contract, and in this YAML file, so renaming a topic means changing both.

The Maven build

A parent POM and five modules: `protocol` ← `engine` ← `server`, and `protocol` ← `client-core` ← `client-tui`, where each arrow means "is used by". The parent does three jobs:

1. **Fixing every version, in one place.** All library versions live in `<dependencyManagement>`: kafka-clients 4.3.0, jackson-databind 2.19.0, slf4j 2.0.17, the JUnit BOM 5.12.2 and the Testcontainers BOM 1.21.3. All of them were checked as the current stable releases on Maven Central when the project started, deliberately avoiding alpha and milestone builds. The modules then declare dependencies without versions, so there is exactly one place where a version can be wrong.

2. **Java 21 through `maven.compiler.release`** — `release` rather than `source` and `target`, so the compiler also checks that the code only uses APIs that exist in JDK 21.
3. **Plugin management**: compiler 3.14.0, surefire/failsafe 3.5.3, exec 3.5.0 — pinned so builds don't drift with Maven defaults.

Module-level choices:

- **Unit tests and integration tests are separated by plugin.** Surefire runs the `*Test` classes during the `test` phase in every module. **Failsafe is switched on only in `server`**, where it runs the `*IT` classes during `verify`. The result: `mvn test` never needs Docker, while `mvn verify` runs the Testcontainers end-to-end test. The failsafe configuration also sets the `api.version=1.44` system property, the workaround for the repackaged docker-java client against Docker daemon 29 or later (ISSUES.md #4), with a comment saying when it can be removed.
- **`slf4j-simple` is declared per logging module** (server, client-core, client-tui): kafka-clients logs through the slf4j API but does not expose it at compile scope (ISSUES.md #6).
- **exec-maven-plugin** is configured with a `mainClass` in `client-tui` (so `mvn -pl client-tui exec:java` just works); the server is launched with an explicit `-Dexec.mainClass=com.goosegame.server.GooseServer`.

The multi-module trap, worth remembering

`mvn -pl engine test` fails on a fresh clone. Maven looks for the `protocol` SNAPSHOT in the local repository, where it has never been installed. Two commands that do work (ISSUES.md #2):

- `mvn -pl <module> -am test` — `-am` ("also make") builds the in-reactor dependencies too;
- for `exec:java` runs, `mvn -q -DskipTests install` once, so sibling SNAPSHOTs exist in the local repository.

Patterns applied

- **Infrastructure as code, and reviewed like code** — the compose file went through the same review process as the Java sources.

- **Environments that fail immediately** — startup order driven by health checks, no topics created automatically, and state that can be thrown away.
- **One place that defines every version** — `dependencyManagement` plus BOMs.

Anti-patterns avoided

- **Depending on SNAPSHOT or "latest" versions** — every version is fixed, plugins included.
- **Letting a replicated cluster create topics by itself** — switched off.
- **Internal topics with too few copies** — offsets and transaction state are also kept in three copies.
- **Ordering containers with sleep commands** — health checks and `depends_on` conditions instead.

Decisions (from DECISIONS.md)

- Failsafe, and therefore the Docker-based tests, runs only in `server`.
- Topic names are written twice, in Java and in YAML; this is a known weak point.
- The build and workflow choices above are recorded there as well.

Issues (from ISSUES.md)

#2 — building one module without `-am` fails on a fresh clone.

#4 — the API version problem between Testcontainers and Docker 29: the environment variable was ignored, overriding the dependency had no effect because the client is repackaged inside Testcontainers, and the fix was the `api.version` system property.

#6 — slf4j is not passed on at compile time, so each module that logs has to declare it.

08 — Testing Strategy

The tests follow the shape of the architecture: the purer a layer is, the more tests it gets and the cheaper those tests are. There are 105 tests in total and no mocking framework anywhere. Every place a test needs to substitute something — `DiceRoller`, `Clock`, `GameListener` — is a small interface, so the test can implement it in one line. See seam.

Module	Tests	Kind	What they prove
protocol	46	unit	Round-trip of every message type; rejection of malformed / unknown-type / oversized / non-string-timestamp payloads; unknown-*field* tolerance; tombstone nulls; validation rules incl. duplicates
engine	39	unit	Every board rule; every rejection; trap/free/turn flow; the deadlock amendment; full scripted games with fixed dice
server	1	E2E (Testcontainers)	The entire stack against a real broker
client-core	10	unit	The view fold over scripted event sequences (replay logic)
client-tui	9	unit	Board layout, markers, pieces, status block, winner line, the sentence written for each event, correct handling of the ANSI codes, and the cell that holds too many players

Why most of the tests are unit tests

This is the usual test pyramid shape, and here is why it falls out that way:

- **Protocol and engine carry most of the weight** because they are pure. Their tests take milliseconds, need nothing running, and can go through every rule one by one. A full scripted game in `GameEngineTest` folds every event through `GameState` exactly as the server would, so most of what people would call "integration behaviour" is already proved below the integration layer.
- **The server gets one test, but that test is the whole system.** What the server adds is the Kafka setup, replay, the order of confirming writes before committing offsets, and shutdown. None of it means anything except against a real broker. Unit-testing it would mean replacing `KafkaConsumer` with a fake, and that tests the fake, not the agreement with Kafka.
- **`mvn test` never needs Docker.** The end-to-end test is an `*IT` class run by failsafe during `verify` (chapter 7), so the logic gives feedback in seconds and the full proof is one command away.

The deterministic E2E

`GooseServerIT` starts a temporary `apache/kafka:4.3.0` container with Testcontainers and creates the topics through the Admin API. It then runs the real `GooseServer.run()` on a virtual thread and plays a two-player game *entirely from the outside*: it writes commands and reads events over the network, and never touches anything inside the server.

Getting the same result every time is designed in, not hoped for:

- **The dice are scripted**, supplied through the same `DiceRoller` seam that production uses. It is a queue read with `queue::remove`, which throws when it runs out, so if the script and the logic ever disagree the test fails clearly instead of continuing with meaningless rolls. The script is chosen so that the game includes a goose hop, the bridge, and an exact landing on 63.
- **The test reacts to events; it does not wait for a fixed time.** For each `GameStarted` or `TurnStarted` event it sends exactly one

`RollDice` for the player named in that event, until `GameWon` arrives. There are no sleeps and no offset arithmetic: the stream of events is what keeps the two sides in step. There is a 90-second deadline, but only to stop a hung test, and its failure message prints the events collected so far.

The checks cover three things:

1. **The opening sequence.** After the joins and the start, whole events are compared for equality.
2. **The landmarks the script was built around**: bob's jump over the bridge, alice's goose hop, and alice's exact win.
3. **The point of event sourcing itself.** Folding the observed history gives the expected final state: `{alice=63, bob=21}`, phase FINISHED, winner alice.

Last, the test checks a clean shutdown. `close()` returns, and the server thread really does stop.

What the automated tests missed and running the real thing found

The most useful testing lesson in this project is issue #7. There were 39 engine tests and a passing end-to-end test, and then the **first game played against the real cluster** froze the system. Two terminal clients, fed from a pipe and rolling once a second — safe, because a roll out of turn is refused and does nothing — reached the well and prison deadlock within a single game. Unit tests check the rules you thought of. Letting the system run freely explores the states you never thought to write a rule about. The fix arrived with two new unit tests, `lastFreePlayerIsNeverTrapped` and `trapStillAppliesWhileAnotherPlayerIsFree`: playing for real finds the problem, unit tests then hold the answer in place.

The same setup of blindly rolling clients became the **final check** at the end of the project (implementation plan, Step 8): a clean `mvn verify`, a fresh cluster, then complete games through the real compose cluster. One of those games was played from the first move to the win with a broker stopped the whole time. Afterwards the copies were confirmed to be in sync again, and a newly started client rebuilt the finished board from the log alone.

That last run taught its own lesson (ISSUES.md #8). The first version of the broker-stopping script waited for `GameStarted`, slept, and then stopped a broker "in the middle of the game". But these games finish in about 90 seconds, so the game had already been *won* before the broker was stopped, and the observation that "moves kept flowing" was true only because there was nothing left to do. The fix was to make the fault a **starting condition** instead of an interruption: stop the broker first, then play the whole game. A fault injected on a timer, against a workload that finishes quickly, can end up testing nothing at all — without saying so.

Testing patterns applied

- **Seams instead of mocks** — `DiceRoller` and `Clock` are passed in; a queue and a fixed clock do the work a mocking framework would.
- **Hold each choice in place with a test.** Some behaviours are decisions, not facts: ignoring unknown fields, passing null tombstones through, the fallback for an overfull cell, cancelling the trap for the last free player. Each of them has a test whose only job is to fail if someone changes the decision without noticing.
- **Test from the outside** — the end-to-end test uses the topics, not the server's methods, and the renderer tests read the produced string, not the renderer's internals.
- **Repeatable by construction** — react to events instead of sleeping, and supply fixed inputs instead of a seeded random generator.
- **Remove the ANSI codes before comparing** — the layout tests match plain text, so changing a colour does not break them, and one separate test checks that the colours are there and are removed cleanly.

Testing anti-patterns avoided

- **Using sleeps to keep two sides in step** — there are none in the suite. The single deadline exists to stop a hung test and to print what it saw, not to wait for something.

- **Faking the very infrastructure under test** — the broker is real wherever the broker's behaviour is the thing being checked.
- **Checking details that are not the agreement** — whole records are compared where the record is the contract, and rendered text is searched with `contains` where the layout is the contract.
- **Trusting a green test run too much** — the layer of games played for real exists precisely because unit tests can only check rules that someone already thought of, and issue #7 proved it.

Issues (from ISSUES.md)

#4 — Testcontainers against Docker daemon 29, fixed with the `api.version` system property in failsafe.

#7 — the deadlock, found by playing the game for real.

#8 — the broker-stopping test that raced the game and was fixed by turning the fault into a starting condition.

Two mistakes in the tests themselves are also worth remembering. One renderer test claimed there was a goose marker on square 55, which is not a goose square: the *test* was wrong and the renderer was right. And a script for the live games once ran `$(date +%s)` separately for each client, which gave the two players different game ids and put them in different games.

09 — Patterns Applied and Anti-Patterns Avoided: the Full List

Everything in one place. Each entry says where the pattern lives in the code and which chapter explains the reasoning. One idea runs through the whole list: **most entries are simply Java 21 language features or Kafka features used the way they were meant to be used**. The project is built on the belief that modern languages have absorbed many patterns that once had to be written by hand.

Architectural patterns

Pattern	Where	Chapter
Event Sourcing	`game.events` as sole source of truth; all state a fold; replay everywhere	01
CQRS, on a small scale	Commands and events on separate topics *and* as separate sealed types; `GameView` is the read model	01, 05
One writer only	Only the server writes `game.events`, and keying by `gameId` means one writer per game	01
Functional core, imperative shell	The pure `engine` inside the `server` that talks to Kafka; the pure `BoardRenderer` inside `Main`, which does the printing	03, 04, 06
Ports and adapters	`GameListener` is the port for a UI; `client-core` knows of no UI at all	05
State machine	A `Phase` enum plus switches covering every case; a move that is not allowed is refused	03
Outbox-style ordering	Confirm the write *first*, commit the offset second — never accept the input before the output is safely stored	04

Design and language patterns: Java 21 doing the work

Pattern	How it is done	Chapter
Algebraic data types	A `sealed interface` with one `record` per case for `Command` and `Event`, and switches with **no `default`**, so adding a message type breaks every fold and every renderer at compile time	02
Parse, don't validate	All checks happen in the compact constructors, so an invalid message cannot exist as an object, no matter where it came from	02
Make invalid states impossible to express	`Optional` fields for "not there"; a `Phase` enum instead of boolean flags; records that cannot be changed at any depth, using `List.copyOf` and `Map.copyOf`	02, 03
Strategy through a functional interface	`DiceRoller` is the one seam that replaces a mocking framework, and `Clock` does the same for time	03, 08
Static factory method	`GameClient.connect()` stops `this` escaping during construction and gives the operation a name	05
Copy methods for building the next record	Explicit `withPosition`, `withStuck` and similar, because Java still has no `with` expression up to version 25	03
Comparing by value as a test tool	The end-to-end test compares whole events with `assertEquals`, which works because the messages are records	08

Concurrency patterns

Pattern	Where	Chapter
All state inside one thread	In the server, one thread owns the consumers, the producer and the state map, so there are no locks at all; the client event loop works the same way	04, 05
Shutdown that signals rather than touches	A `volatile` flag, then `consumer.wakeup()` — the one consumer method that may be called from another thread — then a wait with a time limit; whoever opened a resource closes it	04, 05
Virtual threads for I/O loops	The client event loop, and the server itself in the end-to-end test	05, 08
Publishing an unchangeable value	`volatile GameView view`, which can be read without locking because the value itself never changes	05

Kafka patterns

Pattern	Where	Chapter
Keys to keep one game in order	Every record is keyed by `gameId`	01
At-least-once with a deliberate order of operations	Flush and confirm every write before `commitSync`, with auto-commit off	04
Idempotent producer, `acks=all`, and at least 2 copies in sync	Producer settings in the server and topic settings in the cluster	04, 07
Replay by assigning partitions by hand	No consumer group for the server's startup replay; clients use a new group each run plus `earliest`	04, 05
Stepping past a poison pill	Catch `RecordDeserializationException`, log it, then `seek(offset+1)`	04
Letting tombstones through	The Serde returns null for null, and the code that receives it handles null openly	02

Anti-patterns avoided — and where the temptation was real

Security & robustness:

- **Jackson default typing**, which can end in remote code execution → an explicit `@JsonSubTypes` list on sealed types, so a message can name 11 record types and nothing else, ever.
- **Input with no size limit and no checks** → a 10 KiB limit before parsing, and names limited to `[A-Za-z0-9_-]`, which removes injection into logs, terminals and shells at the boundary.
- **Trusting a client with randomness or state** → the dice are rolled only on the server, with `SecureRandom`.
- **Consumer loops that die on a bad record** → unreadable messages are skipped and are never fatal.

Correctness:

- **Writing the same change to two places** → the local state is built only from exactly the events that were produced.
- **Delivery guarantees nobody states** → the at-least-once gap is documented in the code that creates it, and auto-commit, which would quietly turn this into at-most-once, is switched off on purpose.
- **The same business rule in two places** → the rules live once, in `decide`, and the fold trusts the log. The one repetition that *is* deliberate, the client's fold, is explained, kept in check by the shared protocol tests, and written down: the choice between harmful repetition and a shared abstraction that does not fit was made openly. See coincidental duplication.
- **Rules that can loop for ever** → goose chains are proved to end, and a roll of zero is refused by `Board.resolve`.
- **Using exceptions for normal outcomes** → a refusal is data, an empty list, not a thrown exception.

Concurrency:

- **Sharing changeable state and hoping it works out** → the state stays in one thread, instead of adding locks later.
- **Letting `this` escape from a constructor** → the `connect()` factory starts the thread after the object is built.

- **Ignoring `InterruptedException`** → always set the interrupt flag again, then pass the exception on.
- **Using `volatile` for read-then-write** → `volatile` is used only for flags and for references to values that never change.

Good housekeeping:

- **Methods that return null** → `Optional` or empty collections instead. The two tombstone nulls are the only exception, and they are commented.
- **Losing the cause of an exception** → every wrapper passes the cause on, and messages about unreadable records use `toString()`, because `getMessage()` can be null.
- **Output that depends on the machine's language** → the renderer fixes `Locale.ROOT`.
- **Raw control bytes in source files** → ANSI escapes are written only as `\u001B`. The project hit this twice, once in generated Java and once in these documents; both times a tool introduced them and a person spotted them.
- **Versions drifting over time** → every library and plugin has its version fixed in the parent POM.
- **Using sleeps to keep things in step, in tests and in scripts** → the test driver reacts to events, compose waits on health checks, and since issue #8 a fault is a starting condition rather than something injected on a timer.

Where to read the histories

- `DECISIONS.md` — every choice that was not obvious, grouped by build step, with the reasoning and the conditions under which it should be reconsidered.
- `ISSUES.md` — all 8 problems: what was seen, what was tried and failed, what actually fixed it, and the lesson that carries over to other projects.
- Chapter 11 — the terms used above, each explained in a few lines with a link to a source.

10 — Implementation Plan: the 8-Step Build Order

A multiplayer Game of the Goose (Gioco dell'Oca), built as a test bed for Kafka 4.x and Java 21.

This chapter is the plan the project was actually built from, kept as it was written. It is a working document, so it is written as short checklists rather than as prose: each step lists what to create and how to check it. The other chapters explain the reasoning; this one records the order of the work.

How to use this file (in any working session, with a person or with Claude): each step can be done in one session. Every step stands on its own, ends in a state you can check, and assumes only that the earlier steps are finished. Find the first step that is not ticked, do it, check it, then **stop: show the user every command that was run and all the code that was written, and wait for approval**. Only after approval, tick the step off and commit. Never start the next step without the user saying so.

Status

- ☐ Step 1 — Project scaffolding
- ☐ Step 2 — Kafka cluster (docker compose)
- ☐ Step 3 — `protocol` module (messages + Ser/Des)
- ☐ Step 4 — `engine` module (game rules)
- ☐ Step 5 — `server` module + E2E test
- ☐ Step 6 — `client-core` module
- ☐ Step 7 — `client-tui` module (playable!)
- ☐

Step 8 — README + final verification

Fixed decisions (do not revisit)

Topic	Decision
Stack	Plain Java 21 + `kafka-clients`, no framework
Ser/Des	JSON via Jackson; hand-written `JsonSerde<T>` implementing Kafka's `Serializer`/`Deserializer`; explicit `"type"` field mapped to a closed sealed hierarchy (no polymorphic default typing)
Kafka	Docker Compose, 3 KRaft brokers, topics RF=3, `min.insync.replicas=2`
Build	Maven multi-module, Java 21 (Temurin 21.0.9 installed)
UI	TUI first; `client-core` is UI-agnostic so web/desktop UIs can be added later
Tests	JUnit 5 unit tests everywhere; Testcontainers for the automated E2E
Architecture	Event-sourced, server-authoritative. Topics `game.commands` (intents) and `game.events` (facts), keyed by `gameId`. Clients rebuild state by replaying events.

Library versions checked on Maven Central 2026-07-02: kafka-clients 4.3.0, jackson-databind 2.19.0, slf4j 2.0.x (stable, NOT 2.1.0-alpha), junit-jupiter 5.12.x (stable, NOT 5.13 milestone), testcontainers 1.21.3.

Java 21 idioms to use throughout: records for all messages/state, sealed interfaces + exhaustive pattern-matching `switch`, virtual threads for consumer loops, text blocks, immutable collections, `Instant` timestamps, try-with-resources on every producer/consumer.

Security throughout: validation in record compact constructors; server validates every command against game phase and turn ownership; dice only rolled server-side with `SecureRandom`; deserializer rejects unknown types and caps payload size.

Step 1 — Project scaffolding

Create the Maven multi-module skeleton and put it under git.

- `git init`; `.gitignore` (target/, .idea/, *.iml, .vscode/)

- Parent `pom.xml`: packaging `pom`, Java 21 via `maven.compiler.release`, `<dependencyManagement>` pinning all versions listed above, modules: `protocol`, `engine`, `server`, `client-core`, `client-tui`
- One minimal `pom.xml` per module with only its dependencies:
  - `protocol`: kafka-clients, jackson-databind
  - `engine`: → protocol
  - `server`: → engine; testcontainers (test)
  - `client-core`: → protocol
  - `client-tui`: → client-core
  - all: junit-jupiter (test), slf4j-simple
- Empty `src/main/java` / `src/test/java` trees, base package `com.goosegame`

Verify: `mvn validate` succeeds for all modules. Commit.

Step 2 — Kafka cluster

`docker-compose.yml` at repo root:

- 3 services `kafka-1/2/3`, image `apache/kafka` (pin current tag), KRaft combined controller+broker mode, one shared `CLUSTER_ID`, controller quorum of all three, host ports 9092 / 9094 / 9096
- `init-topics` one-shot service (same image) that waits for the cluster and creates `game.commands` and `game.events`: 3 partitions, replication factor 3, `min.insync.replicas=2`, then exits 0

Verify: `docker compose up -d` → 3 healthy brokers; `kafka-topics.sh --describe` inside a container shows both topics with 3 replicas each and full ISR. Commit.

Step 3 — `protocol` module (messages and their JSON format)

The shared language of the game — the wire contract.

- `Command` sealed interface + records: `JoinGame(gameId, player)`, `StartGame(gameId, player)`, `RollDice(gameId, player)`
- `Event` sealed interface + records: `PlayerJoined`, `GameStarted(players, firstPlayer)`, `TurnStarted(player)`, `DiceR-`

olled(player, die1, die2)`, `PlayerMoved(player, from, to, reason)` (reason: NORMAL/GOOSE/BRIDGE/BOUNCE/MAZE/DEATH…), `PlayerStuck(player, squares)` (inn/well/prison), `PlayerFreed(player)`, `GameWon(player)` — all with `gameId` and `Instant timestamp`
- Validation in compact constructors (non-null, name 1–20 chars `[A-Za-z0-9_-]`)
- `JsonSerde<T>`: implements `Serializer<T>` and `Deserializer<T>` using one shared `ObjectMapper`; writes/reads the `"type"` property via `@JsonTypeInfo(use=NAME)` + `@JsonSubTypes` on the sealed interfaces (closed set — never activate default typing); registers `JavaTimeModule`-free Instant handling or ISO strings; rejects payloads > 10 KB
- Deserialization failure → a `DeserializationException` the server can log-and-skip

Verify: unit tests — round-trip every command/event; unknown `"type"` rejected; malformed JSON rejected; oversized payload rejected. `mvn -pl protocol test` green. Commit.

Step 4 — `engine` module (game rules)

Pure logic, with no Kafka imports at all. This is where the Java 21 features carry the most weight.

- `Board`: 63 squares; geese 5,9,14,18,23,27,32,36,41,45,50,54,59; bridge 6→12; inn 19 (miss one turn); well 31 and prison 52 (stuck until another player lands there); maze 42→39; death 58→back to 1; land exactly on 63 to win, overshoot bounces back
- `GameState` record (immutable): gameId, phase (LOBBY/RUNNING/FINISHED), players in join order, positions, stuck/skip info, current player index. `apply(Event)` returns new state
- `GameEngine.decide(GameState, Command, DiceRoller) -> List<Event>` — pure function; `DiceRoller` is an interface so tests inject fixed rolls and the server injects SecureRandom
- Rejections (out-of-turn roll, join after start, duplicate name, 2–6 players…) produce no events (or a `CommandRejected` event — decide during implementation, keep it simple)

Verify: unit tests for every rule listed above + full simulated game with scripted dice. `mvn -pl engine test` green. Commit.

Step 5 — `server` module and the end-to-end test

The one process allowed to decide what happens.

- `GooseServer.main`: on startup, replay `game.events` from beginning to rebuild a `Map<String, GameState>`; then consumer loop (group `goose-server`) on `game.commands`: for each command → `GameEngine.decide` → produce resulting events to `game.events` (keyed by gameId) → apply events to local state
- Producer configured with `acks=all`, `enable.idempotence=true`
- Dice: `RandomGenerator` backed by `SecureRandom`
- Graceful shutdown hook closing consumer/producer
- **E2E test (Testcontainers):** start throwaway Kafka container, create topics, run server loop on a virtual thread, produce scripted commands for a 2-player game, consume `game.events` and assert the expected sequence and winner

Verify: `mvn -pl server verify` green (includes E2E). Manual: `docker compose up -d`, run server against localhost:9092, send a JoinGame with `kafka-console-producer`, see the PlayerJoined event with `kafka-console-consumer`. Commit.

Step 6 — `client-core` module

A client library that knows nothing about any user interface — the boundary that makes it possible to add a web or desktop UI later.

- `GameClient` (Closeable): producer to send `Command`s; event consumer on a **virtual thread** with a unique consumer group, reading `game.events` from earliest (replay)
- `GameView`: client-side fold of events into displayable state (board positions, whose turn, log of last events, winner)
- `GameListener` interface: `onViewUpdated(GameView)`, `onEvent(Event)` — any UI implements this
- No console I/O in this module

Verify: unit tests — folding a scripted event sequence produces the expected view (replay logic). `mvn -pl client-core test` green. Commit.

Step 7 — `client-tui` module (the game becomes playable)

- `Main`: args = bootstrap servers, gameId, player name (with sensible defaults)
- Draws the 63-square board as a snaking grid in ANSI colour, built from a text block, with each player shown as a coloured letter and the special squares marked; it prints the whole board again on every update
- Stdin loop: `join <name>`, `start`, `roll`, `help`, `quit`
- Runnable via `mvn -pl client-tui exec:java` (add exec plugin) or a small `run-client.sh`

Verify: with compose cluster + server running, two terminals play a full game to victory; killing and restarting a client mid-game reconstructs the board from replay. Commit.

Step 8 — README + final verification

- README: what it is, architecture diagram, quickstart (compose up → server → 2 clients), command reference, and **Kafka experiments**: kill a broker mid-game (RF=3 survives), inspect consumer groups/offsets, replay a finished game, console-consume `game.events` live; optional advanced exercise: enable SASL/SCRAM + ACLs
- Full pass: `mvn verify` green from clean; fresh `docker compose up -d`; complete manual game; broker-kill resilience check

Verify: everything above. Commit. Project done — future ideas: web UI (Javalin + SSE) on top of client-core, Avro + Schema Registry migration, multiple concurrent games.

11 — Glossary: the Terms This Documentation Uses

The chapters use the vocabulary of their fields without stopping to define it. This is that reference: each term in a few lines, with a link to the source that treats it properly.

It is a convenience, not a tutorial. The chapters assume you are reading for the design rather than for an introduction to Kafka, so nothing here has to be read first and nothing here is explained twice. It is simply quicker to settle a word in two lines than to break off and go looking for it.

The terms themselves stay in the chapters unchanged. They are the words these communities use, and paraphrasing them would only make the documentation harder to match against the literature.

Kafka terms

Topic, partition, key

A **topic** is a named log of messages. Each topic is split into **partitions**, and a partition is an ordered, append-only sequence. A message carries a **key**; all messages with the same key go to the same partition, so Kafka guarantees their relative order. This project keys both topics by `gameId`, so one game is always one ordered story.

Source: Kafka: introduction

Offset

The position of a message inside its partition, counted from zero. A consumer stores the offset it has reached, so it can stop and continue later from the same place.

Source: Kafka documentation, section “Consumer Position”

Consumer group

A set of consumers that share the work of reading a topic: each partition goes to exactly one member of the group. Two consumers in *different* groups both receive every message, which is why every client in this project reads the full event log while the server reads each command once.

Source: Kafka documentation, section “Consumers”

Replay

Reading a topic again from the beginning instead of from the last stored offset. Because the log keeps every event, a process that replays it rebuilds its state exactly as it was. This is how the server and every client recover after a restart.

Replication factor, ISR, and minimum in-sync replicas

The **replication factor** is the number of brokers that hold a copy of each partition (3 here). The **ISR**, or *in-sync replicas*, is the subset of those copies that are currently up to date. `min.insync.replicas` (2 here) is the number of in-sync copies a write needs before it is accepted, so the cluster keeps working when one broker is lost but never accepts a write that only one broker has seen.

Source: Kafka documentation, section “Replication”

KRaft

The consensus protocol Kafka uses to manage its own metadata since it stopped depending on ZooKeeper. A KRaft cluster needs no second system to run.

Source: Kafka documentation, section “KRaft”

Delivery semantics: at-most-once, at-least-once, exactly-once

Three levels of guarantee. **At-most-once**: a message may be lost, never handled twice. **At-least-once**: a message is never lost, but a crash at the wrong moment can make it be handled twice. **Exactly-once**: neither, which costs transactions and extra machinery. This

project chooses at-least-once and makes the repeated work harmless.

Source: Kafka documentation, section “Message Delivery Semantics”

Idempotent producer

A producer setting (`enable.idempotence=true`) that lets Kafka recognise and drop a message the producer sent twice after a network retry. It removes duplicates caused by *retries*; it does not remove duplicates caused by the application sending the same thing twice.

Source: Kafka documentation, section “Producer Configs”

Tombstone

A message with a key and a `null` value. In a compacted topic it marks the key as deleted. The word also describes any `null` used to mean “this field is deliberately absent”, which is how this project’s `Serde` treats two optional fields.

Source: Kafka documentation, section “Log Compaction”

Poison pill

A message that the consumer cannot deserialize. Without a plan, it stops the consumer forever: the consumer fails, restarts, reads the same message, and fails again. The fix is to catch the failure, log it, and skip past that offset.

Source: Error handling patterns for Kafka applications (Confluent)

Serde

Short for *serializer / deserializer*: the pair of functions that turn an object into bytes for Kafka and back again.

Design and architecture terms

Event sourcing

Storing the history of *what happened* as an ordered list of events, and deriving the current state from that history, instead of storing the current state and overwriting it. The log is the truth; everything else can be thrown away and rebuilt.

Source: Event sourcing (Martin Fowler)

Fold

Walking through a list and combining its items one at a time into a single result, carrying the result forward at each step. Adding up a list of numbers is a fold. In this project, `state.apply(event)` is applied to every event in order, and the result is the current game state.

Source: Fold (higher-order function)

Server-authoritative

Only the server is allowed to decide what happens. Clients send *requests*, not *results*, so a modified or hostile client cannot invent a dice roll or move a piece. It can only ask.

Wire contract

The exact shape of the messages that travel between processes: the field names, the types, and the rules for reading them. It is a contract because both sides must agree on it, and it can only be changed carefully once other programs depend on it.

CQRS and the read model

CQRS separates the code that *changes* state from the code that *reads* it. The **read model** is a copy of the state shaped for display, rebuilt from the same events. In this project `GameView` is a read model: it is only ever shown, never used to decide anything.

Source: CQRS (Martin Fowler)

Ports and adapters (hexagonal architecture)

The core of the program defines the *ports* it needs as interfaces, and the outside world is plugged in through *adapters* that implement them. The core never knows which adapter it is talking to. Here `GameListener` is the port and the terminal UI is one adapter.

Source: Hexagonal architecture (Alistair Cockburn)

Functional core, imperative shell

A design where all the rules live in pure functions that only compute values, and everything impure — reading input, writing output, talking to the network — is pushed out to a thin outer layer. The core is easy to test because it does nothing but return values.

Source: Boundaries (Gary Bernhardt, talk)

Parse, don’t validate

Instead of checking that data is valid and then passing the same loose type around, convert it once into a type that *cannot* hold invalid data. After the conversion, no further checking is needed anywhere. Here the record constructors do the parsing.

Source: Parse, don’t validate (Alexis King)

Coincidental duplication

Two pieces of code that look the same today but exist for different reasons and will change for different reasons. Merging them creates a shared abstraction that both sides then fight against. This is why the client rebuilds the game state with its own fold instead of reusing the engine’s.

Source: The wrong abstraction (Sandi Metz)

Seam

A place in a program where you can change what happens without editing the code around it — typically by passing in a different implementation. Seams are what make code testable without a

mocking framework. `DiceRoller` is a seam: the test passes a fixed sequence of rolls.

Source: Legacy seam (Martin Fowler)

Java terms

Record

A class whose whole job is to hold values. Java writes the constructor, accessors, `equals`, `hashCode` and `toString` for you, and the fields cannot change after construction.

Source: JEP 395: Records

Compact constructor

A short form of a record's constructor that only contains the checks and adjustments, without listing or assigning the fields. It runs before the fields are set, so it is the natural place to reject invalid values.

Sealed interface

An interface that names exactly which types are allowed to implement it. The compiler then knows the full list, which is what makes a `switch` over those types provably complete.

Source: JEP 409: Sealed classes

Exhaustive pattern matching

A `switch` over a sealed type that must cover every case. If a new type is added and some `switch` does not handle it, compilation fails — the compiler finds the gap instead of the user.

Source: JEP 441: Pattern matching for switch

Virtual thread

A thread managed by the JVM instead of the operating system. Virtual threads are cheap enough to create one per task and let it block, instead of writing callback-style code.

Source: JEP 444: Virtual threads

Testing terms

Test pyramid

The rule of thumb that a project should have many fast unit tests, fewer service-level tests, and very few slow end-to-end tests. The shape follows from cost: the tests at the top are the slowest to run and the most annoying to maintain.

Source: The practical test pyramid (Martin Fowler)

Testcontainers

A library that starts real services in Docker containers from inside a test and shuts them down afterwards. It is how the end-to-end test here runs against a real Kafka broker rather than a fake one.

Source: Testcontainers

Smoke test

A quick, manual run of the real system to see whether it works at all, as opposed to an automated test that checks one specific rule. Several of the problems in `ISSUES.md` were found this way and only afterwards pinned down by a unit test.